DEOS Network & System Management ICD

Sürüm 1.4 – SYSTEM Servisi, Ağ Yönetimi, Düğüm Bilgisi ve Telemetri/Değişim Aboneliği

Emir Arkalı (emirarkali07@gmail.com)
DEOS – Dokuz Eylül Otonom Sistemler

İlk Yayın: 28 Eylül 2026 | Son Revizyon: 15 Ekim 2026

Doküman Tipi:	Interface Control Document (ICD)	**Doküman Kodu:**	`DOC-ICD-NET-003`
Doküman Sürümü:	Sürüm 1.4	**Protokol Sürümü:**	v1.1 (`0x11`) – Medya Bağımsız
Yazar:	Emir Arkalı (`emirarkali07@gmail.com`)	**Son Değiştiren:**	Emir Arkalı
İlk Yayın Tarihi:	28 Eylül 2026	**Son Revizyon Tarihi:**	15 Ekim 2026
Durum:	Yürürlükte / Onaylandı	**Taşıma / Kapsam:**	SYSTEM Servisi (`0x00`), PING/PONG, Node Info & Telemetri/Olay Aboneliği

Doküman Revizyon Geçmişi

Revizyon	Tarih	Yazar / Değiştiren	Açıklama
v0.3	28 Eylül 2026	Emir Arkalı	İlk taslak ve monolitik ICD v0.3 tanımı.
v1.0	04 Ekim 2026	Emir Arkalı	Ağ yönetimi bağımsız doküman oluşturuldu.
v1.1	13 Ekim 2026	Emir Arkalı	Sürüm 1.1 standardizasyonu, kurumsal üstveri ve revizyon geçmişi eklendi.
v1.2	15 Ekim 2026	Emir Arkalı	Düğüm bilgisi sorgulama (GET_NODE_INFO) istek ve 20 baytlık standart cevap formatı eklendi.
v1.3	15 Ekim 2026	Emir Arkalı	Dinamik telemetri ve durum aboneliği (SUBSCRIBE/UNSUBSCRIBE) mekanizması ve çoklu abone kuralları eklendi.
v1.4	15 Ekim 2026	Emir Arkalı	Değişim bazlı (On-Change / Event-Driven) telemetri abonelik modu (`interval_ms = 0xFFFF`) tanımlandı.

1 SYSTEM Service Komutları

ID	Symbol
0x01	DEOS_CMD_SYSTEM_GET_STATE
0x02	DEOS_CMD_SYSTEM_SET_STATE
0x03	DEOS_CMD_SYSTEM_GET_MODE
0x04	DEOS_CMD_SYSTEM_SET_MODE
0x05	DEOS_CMD_SYSTEM_RESET_NODE
0x06	DEOS_CMD_SYSTEM_GET_NODE_INFO
0x07	DEOS_CMD_SYSTEM_HEARTBEAT
0x08	DEOS_CMD_SYSTEM_PING
0x09	DEOS_CMD_SYSTEM_PONG
0x0A	DEOS_CMD_SYSTEM_SUBSCRIBE
0x0B	DEOS_CMD_SYSTEM_UNSUBSCRIBE
0x0C	DEOS_CMD_SYSTEM_GET_SUBSCRIPTIONS

```
typedef enum
{
    DEOS_CMD_SYSTEM_GET_STATE          = 0x01,
    DEOS_CMD_SYSTEM_SET_STATE          = 0x02,
    DEOS_CMD_SYSTEM_GET_MODE           = 0x03,
    DEOS_CMD_SYSTEM_SET_MODE           = 0x04,
    DEOS_CMD_SYSTEM_RESET_NODE         = 0x05,
    DEOS_CMD_SYSTEM_GET_NODE_INFO      = 0x06,
    DEOS_CMD_SYSTEM_HEARTBEAT          = 0x07,
    DEOS_CMD_SYSTEM_PING               = 0x08,
    DEOS_CMD_SYSTEM_PONG               = 0x09,
    DEOS_CMD_SYSTEM_SUBSCRIBE          = 0x0A,
    DEOS_CMD_SYSTEM_UNSUBSCRIBE        = 0x0B,
    DEOS_CMD_SYSTEM_GET_SUBSCRIPTIONS  = 0x0C
} deos_system_command_t;
```

2 PING–PONG Ağ Erişilebilirlik Mekanizması

DEOS protokolünde tüm adreslenebilir physical ve virtual node'lar ortak bir PING–PONG mekanizması uygular. Bir node, erişmek istediği başka bir node'a unicast PING gönderebilir. Geçerli PING mesajını alan hedef node, aynı Ping ID değerini içeren PONG mesajını kaynak node'a geri göndermek zorundadır.

PING ve PONG uygulama REQUEST/RESPONSE mekanizmasından ayrı bir network-management mekanizmasıdır. Her iki mesaj da aşağıdaki alanları kullanır:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
```

PING için `Command = DEOS_CMD_SYSTEM_PING`; PONG için `Command = DEOS_CMD_SYSTEM_PONG` kullanılır.

2.1 PING Mesaj Formatı

PING mesajının uygulama veri alanı aşağıdaki gibi tanımlanır:

Byte	Alan	Tip	Açıklama
0	Protocol Version	`uint8_t`	DEOS v1.1 için `0x11`
1	Sequence	`uint8_t`	Gönderen node'un TX sequence değeri
2	Command	`uint8_t`	`DEOS_CMD_SYSTEM_PING` = `0x08`
3	Length	`uint8_t`	`0x04`
4–7	Ping ID	`uint32_t`	Gönderen tarafından üretilen correlation değeri

Toplam semantic CAN-FD data length 8 byte'tır. Ping ID, DEOS genel byte-order kuralına uygun olarak Little-Endian taşınır. Gönderen node, aynı anda birden fazla PING işlemi yürütüyorsa aktif işlemleri ayırt edecek Ping ID değerleri kullanmalıdır.

PING CAN Identifier alanları:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
Destination   = ping atılan node
Source        = ping gönderen node
```

2.2 PONG Mesaj Formatı

PONG mesajı, alınan PING mesajındaki 32-bit Ping ID değerini değiştirmeden geri taşır.

Byte	Alan	Tip	Açıklama
0	Protocol Version	`uint8_t`	DEOS v1.1 için `0x11`
1	Sequence	`uint8_t`	PONG gönderen node'un kendi TX sequence değeri
2	Command	`uint8_t`	`DEOS_CMD_SYSTEM_PONG` = `0x09`
3	Length	`uint8_t`	`0x04`
4–7	Ping ID	`uint32_t`	PING'den aynen echo edilen correlation değeri

PONG CAN Identifier alanları:

```
Priority      = DEOS_PRIO_NETWORK
Message Class = DEOS_CLASS_NETWORK
Service       = DEOS_SERVICE_SYSTEM
Destination   = PING mesajının Source Node değeri
Source        = PING'i alan hedef node
```

PONG mesajındaki Sequence alanı PING mesajındaki Sequence değerini kopyalamaz. Her node kendi transmit sequence sayacını kullanmaya devam eder. PING ile PONG arasındaki transaction correlation yalnızca Ping ID üzerinden yapılır.

2.3 Zorunlu Node Davranışı

Geçerli ve kendisine yöneltilmiş bir PING alan her DEOS node'u aşağıdaki davranışı uygular:

1. PING mesajının Source Node ve Ping ID alanlarını okur.
2. Yeni bir PONG mesajı oluşturur.
3. PONG Source alanını kendi Node ID'si yapar.
4. PONG Destination alanını PING Source değerine eşitler.
5. PING içerisindeki Ping ID değerini değiştirmeden PONG payload'ına kopyalar.
6. Kendi normal TX Sequence değerini kullanarak PONG'u gönderir.

2.4 RTT Ölçümü

Round-trip time ölçümü gönderen node üzerinde yapılır. PING gönderildiği anda yerel zaman kaydedilir; aynı Ping ID'yi taşıyan PONG alındığında iki yerel zaman arasındaki fark RTT olarak hesaplanır:

$$RTT = T_{pong} - T_{ping}$$

Bu yöntem node'lar arasında saat senkronizasyonu gerektirmez. Timeout süresi transport veya uygulama gereksinimine göre ayrıca tanımlanabilir; v0.3 kapsamında tek bir global timeout süresi dondurulmamıştır.

2.5 Broadcast PING Kuralı

`DEOS_CMD_SYSTEM_PING` mesajı v1.1 protokolünde yalnızca unicast Destination Node ile kullanılmalıdır. `Destination = DEOS_NODE_GLOBAL_BROADCAST` veya `DEOS_NODE_CAN_BROADCAST` ile PING gönderimi geçersizdir. Bu kural, çok sayıda node'un aynı anda PONG üretmesinden doğacak response burst trafiğini önlemek için tanımlanmıştır.

Bir uygulama tüm node'ların erişilebilirliğini taramak istiyorsa her hedef Node ID için ayrı unicast PING işlemi başlatmalıdır.

2.6 HEARTBEAT Mesaj Formatı

HEARTBEAT, bir node'un periyodik veya olay tabanlı olarak kendi yaşam bilgisini yayınlamasıdır. PING–PONG ise belirli bir node'un başka belirli bir node'a aktif erişilebilirlik sorgusu yapmasıdır. Bu iki mekanizma birbirinin yerine kullanılmaz.

v1.1 protokolünde HEARTBEAT mesajı boş (0 byte payload) değildir. Node'un anlık durumunu (State) taşıyan 1 byte'lık bir payload barındırır:

Byte	Alan	Tip	Açıklama
0	Current State	`uint8_t`	`deos_state_t` enum değeri (örn: `0x03` ACTIVE, `0x05` SAFE)

3 Düğüm Bilgisi (GET_NODE_INFO) Sorgulama Mekanizması

DEOS protokolünde yer alan bir düğümün donanım tipi, firmware revizyonu, protokol versiyonu, aktif çalışma süresi (uptime) ve desteklediği servis haritasını (capabilities bitmask) sorgulamak için `DEOS_CMD_SYSTEM_GET_NODE_INFO` (`0x06`) komutu kullanılır.

Bu mekanizma, özellikle telemetri bilgisayarı (Raspberry Pi / Textual UI), Yer Kontrol İstasyonu ve ROS 2 Köprüsü (`deos_ros_bridge`) tarafından ağa yeni bağlanan veya hatta bulunan düğümlerin durumunu ve sürüm uyumluluğunu doğrulamak için kullanılır.

3.1 İstek (REQUEST) Mesaj Formatı

Düğüm bilgisi sorgulama işlemi bir istek–cevap (REQUEST–RESPONSE) işlemidir. İstek mesajı aşağıdaki başlık alanlarını kullanır:

```
Priority      = DEOS_PRIO_NETWORK (0x05) veya DEOS_PRIO_LOW (0x06)
Message Class = DEOS_CLASS_REQUEST (0x04)
Service       = DEOS_SERVICE_SYSTEM (0x00)
Command       = DEOS_CMD_SYSTEM_GET_NODE_INFO (0x06)
```

Byte	Alan	Tip	Açıklama
0	Protocol Version	`uint8_t`	DEOS v1.1 için `0x11`
1	Sequence	`uint8_t`	İstek yapan node'un TX sequence değeri
2	Command	`uint8_t`	`DEOS_CMD_SYSTEM_GET_NODE_INFO` = `0x06`
3	Length	`uint8_t`	`0x00` (Ek payload verisi bulunmaz)

3.1.1 Unicast Zorunluluğu

Tıpkı PING mekanizmasında olduğu gibi, `DEOS_CMD_SYSTEM_GET_NODE_INFO` istekleri yalnızca **Unicast** hedef adreslerle (`Destination != DEOS_NODE_GLOBAL_BROADCAST` ve `Destination != DEOS_NODE_CAN_BROADCAST`) gönderilmelidir. Broadcast olarak atılan istekler, özellikle CAN-FD ve LoRa hattında aynı anda çok sayıda düğümün cevap üretmesine (response burst/collision) ve bant genişliğinin kilitlenmesine neden olacağı için kabul edilmez.

3.2 Cevap (RESPONSE) Mesaj Formatı

Sorgulanan hedef düğüm, geçerli bir istek aldığında `DEOS_CLASS_RESPONSE` sınıfında 20 baytlık standart bir cevap üretir.

Cevap başlık alanları:

```
Priority      = DEOS_PRIO_NETWORK (0x05)
Message Class = DEOS_CLASS_RESPONSE (0x05)
Service       = DEOS_SERVICE_SYSTEM (0x00)
Command       = DEOS_CMD_SYSTEM_GET_NODE_INFO (0x06)
Destination   = İstek gönderen kaynak node ID
Source        = Sorgulanan yerel node ID
```

Cevap veri yükü (Payload) anatomisi (Toplam 20 Byte, Little-Endian):

Byte	Alan Adı	Tip	Boyut	Açıklama
0	Result Code	`uint8_t`	1 Byte	İşlem sonucu: `0x00` SUCCESS (`deos_result_t`)
1	Node ID	`uint8_t`	1 Byte	Cevap veren düğümün mantıksal kimliği (`deos_node_id_t`)
2	Hardware Type	`uint8_t`	1 Byte	Donanım tipi (`0x00` UNKNOWN, `0x01` STM32H5, `0x02` STM32F4, `0x03` LINUX_PC, `0x04` ESP32)
3	Protocol Version	`uint8_t`	1 Byte	Desteklenen wire format sürümü (`0x11` = v1.1)
4	FW Major	`uint8_t`	1 Byte	Firmware ana sürümü (Örn: 1)
5	FW Minor	`uint8_t`	1 Byte	Firmware alt sürümü (Örn: 2)
6	FW Patch	`uint8_t`	1 Byte	Firmware yama sürümü (Örn: 0)
7	Reserved	`uint8_t`	1 Byte	4-bayt hizalama için dolgu baytı (`0x00`)
8–11	Git Hash	`uint32_t`	4 Byte	Derlemenin Git commit hash'inin ilk 4 baytı
12–15	Supported Services	`uint32_t`	4 Byte	Desteklenen servislerin bitmask değeri
16–19	Uptime	`uint32_t`	4 Byte	Düğümün açılışından bu yana geçen süre (Saniye)

3.3 Cevap Veri Yapısı (C Struct Tanımı)

```
typedef struct
{
    uint8_t  result_code;        /* 0x00: SUCCESS (deos_result_t) */
    uint8_t  node_id;            /* Logical Node ID (deos_node_id_t) */
    uint8_t  hw_type;            /* Hardware Type (deos_hw_type_t) */
    uint8_t  proto_version;      /* Protocol Version (0x11 = v1.1) */
    uint8_t  fw_version_major;   /* Firmware Version - Major */
    uint8_t  fw_version_minor;   /* Firmware Version - Minor */
    uint8_t  fw_version_patch;   /* Firmware Version - Patch */
    uint8_t  reserved;           /* Alignment padding (0x00) */
    uint32_t git_hash;           /* Git commit hash prefix (4 bytes) */
    uint32_t supported_services; /* Bitmask: (1 << DEOS_SERVICE_SYSTEM) | ... */
    uint32_t uptime_s;           /* Node uptime in seconds */
} __attribute__((packed)) deos_node_info_payload_t;
```

3.4 Hardware Type Değerleri (`hw_type`)

`hw_type` 8-bit alanı, sorgulanan düğümün üzerinde çalıştığı mikrodenetleyici veya bilgisayar platformunu tanımlar:

Değer	C Sabiti	Platform	Açıklama
`0x00`	`DEOS_HW_TYPE_UNKNOWN`	Tanımsız / Bilinmeyen	Donanım tipi henüz tanıtılmamış
`0x01`	`DEOS_HW_TYPE_STM32H5`	STM32H5 Serisi	NUCLEO-H563ZI vb. Cortex-M33 ana işlemciler
`0x02`	`DEOS_HW_TYPE_STM32F4`	STM32F4 Serisi	NUCLEO-F446RE vb. Cortex-M4 yardımcı düğümler
`0x03`	`DEOS_HW_TYPE_LINUX_PC`	Linux Tabanlı Bilgisayar	ROS 2 Bilgisayarı / Raspberry Pi / Jetson
`0x04`	`DEOS_HW_TYPE_ESP32`	ESP32 Serisi	Kablosuz / WiFi / BLE yardımcı telemetri

3.5 Supported Services Bitmask Alanı

`supported_services` 32-bit bitmask alanı, sorgulanan düğümün hangi fonksiyonel servisleri barındırdığını belirtir:

Bit	Servis Karşılığı	Değer
Bit 0	DEOS_SERVICE_SYSTEM	1 « 0x00 (0x00000001)
Bit 1	DEOS_SERVICE_TRACTION	1 « 0x01 (0x00000002)
Bit 2	DEOS_SERVICE_STEERING	1 « 0x02 (0x00000004)
Bit 3	DEOS_SERVICE_BRAKE	1 « 0x03 (0x00000008)
Bit 4	DEOS_SERVICE_BMS	1 « 0x04 (0x00000010)
Bit 5	DEOS_SERVICE_CALIBRATION	1 « 0x05 (0x00000020)
Bit 6	DEOS_SERVICE_DIAGNOSTIC	1 « 0x06 (0x00000040)
Bit 7	DEOS_SERVICE_SENSORS	1 « 0x07 (0x00000080)
Bit 8	DEOS_SERVICE_LIGHTING	1 « 0x08 (0x00000100)

4 Dinamik Telemetri ve Durum Abonelik Mekanizması (Subscription Engine)

DEOS ağında düğümlerin veri yolunu sürekli GET_STATUS istekleriyle (Polling) meşgul etmesini veya hatta dinleyen bir alıcı olmasa dahi körü körüne periyodik yayın (Blind Push) yapmasını önlemek amacıyla dinamik bir **Telemetri ve Durum Abonelik Mekanizması (On-Demand Telemetry Subscription)** tanımlanmıştır.

Bu mekanizma sayesinde bir tüketici düğüm (Subscriber; örneğin DEOS_NODE_MAIN_STM32 veya DEOS_NODE_ROS_BRIDGE), veri üreten bir düğüme (Publisher; örneğin DEOS_NODE_BMS_MAIN veya DEOS_NODE_TRACTION) tek bir abonelik isteği göndererek, istenen durum verisinin belirlenen periyotta ve belirlenen kira süresi (TTL) boyunca kendisine otomatik ve tek yönlü akmasını sağlar.

4.1 SUBSCRIBE İstek Mesajı (REQUEST)

Abonelik başlatmak veya mevcut bir aboneliğin periyodunu/kira süresini güncellemek için DEOS_CMD_SYSTEM_SUBSCRIBE (0x0A) komutu kullanılır.

Mesaj Başlık Alanları:

```
Priority      = DEOS_PRIO_CONFIG (0x04) veya DEOS_PRIO_NETWORK (0x05)
Message Class = DEOS_CLASS_REQUEST (0x04)
Service       = DEOS_SERVICE_SYSTEM (0x00)
Command       = DEOS_CMD_SYSTEM_SUBSCRIBE (0x0A)
Destination   = Hedef Üretici Düğüm (Unicast)
Source        = İstek Yapan Abone Düğüm
```

İstek Veri Yükü (Payload) Anatomisi (Toplam 6 Byte, Little-Endian):

Byte	Alan Adı	Tip	Boyut	Açıklama
0	Target Service	uint8_t	1 Byte	Abone olunmak istenen servis (DEOS_SERVICE_BMS, SYSTEM vb.)
1	Target Command	uint8_t	1 Byte	Abone olunmak istenen durum komutu (DEOS_CMD_BMS_STATUS, HEARTBEAT vb.)
2–3	Interval	uint16_t	2 Byte	İstenen periyot (milisaniye, ms): 0 = İptal, 10-65534 = Periyodik akış, 0xFFFF (DEOS_INTERVAL_ON_CHANGE) = Sadece Durum Değiştiğinde Gönder (On-Change Modu).
4–5	Lease TTL	uint16_t	2 Byte	Kira Süresi (Saniye): 0 = Sonsuz (Manuel iptale kadar). > 0 = Abone canlı kalmaz veya yenilemezse otomatik kapanma süresi.

4.2 SUBSCRIBE Cevap Mesajı (RESPONSE)

Üretici düğüm, gelen abonelik isteğini işler ve sonucunu aboneye bildirir:

Mesaj Başlık Alanları:

```
Priority      = DEOS_PRIO_NETWORK (0x05)
Message Class = DEOS_CLASS_RESPONSE (0x05)
Service       = DEOS_SERVICE_SYSTEM (0x00)
Command       = DEOS_CMD_SYSTEM_SUBSCRIBE (0x0A)
Destination   = İstek yapan abone node ID
Source        = Cevap veren üretici node ID
```

Cevap Veri Yükü (Payload) Anatomisi (Toplam 5 Byte, Little-Endian):

Byte	Alan Adı	Tip	Boyut	Açıklama
0	Result Code	`uint8_t`	1 Byte	İşlem sonucu: `0x00` SUCCESS, `0x01` UNSUPPORTED, `0x03` OUT_OF_RANGE, `0x05` BUSY (`deos_result_t`)
1	Target Service	`uint8_t`	1 Byte	Onaylanan hedef servis ID
2	Target Command	`uint8_t`	1 Byte	Onaylanan hedef komut ID
3–4	Actual Interval	`uint16_t`	2 Byte	Üretici tarafından tahsis edilen gerçek periyot (ms). (Donanım çözünürlüğüne yuvarlanmış değer).

4.3 UNSUBSCRIBE İstek ve Cevap Mesajı

Aktif bir aboneliği sonlandırmak için `DEOS_CMD_SYSTEM_UNSUBSCRIBE` (`0x0B`) komutu kullanılır.

İstek Veri Yükü (Payload) Anatomisi (Toplam 2 Byte):

Byte	Alan Adı	Tip	Boyut	Açıklama
0	Target Service	`uint8_t`	1 Byte	İptal edilecek servis ID
1	Target Command	`uint8_t`	1 Byte	İptal edilecek durum komutu ID. (`0xFF` değeri o servisteki veya tüm aktif abonelikleri topluca kapatır).

Cevap formatı standart DEOS generic response yapısındadır (`Byte 0: Result Code = 0x00 SUCCESS`).

4.4 Akış Veri Mesajı Formatı (Streaming Data Delivery)

Abonelik onaylandıktan sonra üretici düğüm, belirlenen periyotta veriyi tek yönlü olarak pompalar:

- **Destination:** Abone Düğüm Kimliği (`Subscriber Node ID` – Unicast).
- **Message Class:** `DEOS_CLASS_STATUS` (`0x02`) veya Heartbeat için `DEOS_CLASS_NETWORK` (`0x07`).
- **Service / Command:** Abone olunan orijinal servis ve durum komutu (Örn: `BMS` / `STATUS`).
- **Payload:** İlgili durum komutunun ilgili ICD şartnamesinde tanımlanmış olan standart veri yükü (Örn: `struct deos_bms_status_payload`).

Bu sayede veri akışı için yeni bir taşıma zarfı veya ek bayt yükü getirilmez; sistemde tanımlı telemetri mesajları doğrudan kullanılır.

4.5 Çoklu Abone Mimarisi (Multi-Subscriber Architecture)

DEOS dağıtık yapısında aynı durum verisine birden fazla bağımsız düğüm aynı anda ihtiyaç duyabilir. Örneğin batarya durumuna (`DEOS_SERVICE_BMS` / `DEOS_CMD_BMS_STATUS`):

- `DEOS_NODE_MAIN_STM32` genel araç yönetimi ve ekran için **100 ms (10 Hz)** ile,
- `DEOS_NODE_TRACTION` motor sürücü anlık akım sınırlaması için **20 ms (50 Hz)** ile,
- `DEOS_NODE_ROS_BRIDGE` telemetri kaydı için **1000 ms (1 Hz)** ile

aynı anda bağımsız olarak abone olabilir.

Üretici düğüm bu senaryoyu aşağıdaki kurallarla yönetir:

1. **Bağımsız Abonelik Yuvaları (Independent Slots):** Üretici düğümün statik tablosunda her abone için ayrı bir kayıt açılır. Her abonenin hedef adresi, talep ettiği periyot (`interval_ms`), son gönderim zamanı (`last_sent_ms`) ve kira bitiş süresi (`expire_time_ms`) bağımsız işletilir.
2. **Unicast İletim ve Çoklu Taşıma İzolasyonu:** Her aboneye kendi zamanlayıcısı dolduğunda doğrudan Unicast hedef adresle mesaj iletilir. Bu kural, DEOS Multi-Transport mimarisi için hayati öneme sahiptir: `Main STM32` ve `Traction` CAN omurgasında yer alırken, `ROS Bridge` Ethernet üzerinde, `Ground Control` ise LoRa üzerindedir. Unicast gönderim sayesinde Router, paketi yalnızca ilgili arayüze yönlendirir; dar bantlı LoRa kanalına veya gereksiz alt ağlara sızma (broadcast kirliliği) engellenir.
3. **Bağımsız İptal ve Yaşam Döngüsü:** Bir abonenin aboneliğini iptal etmesi (`UNSUBSCRIBE`) veya kira süresinin (TTL) dolması, diğer abonelerin veri akışını kesinlikle etkilemez.
4. **Kapasite ve Kaynak Yönetimi:** Üretici düğümlerde statik tablo boyutu nominal olarak 8 yuva (`DEOS_MAX_SUBSCRIPTI = 8`) ile sınırlandırılmıştır. Tablo tamamen dolu olduğunda gelen yeni abonelik talebine üretici `DEOS_RESULT_BUSY` (`0x05`) yanıtı döner.

4.6 İsteğe Bağlı Canlılık Bildirimi (On-Demand Heartbeat)

Sistemde bulunan tüm mikrodenetleyicilerin hatta sürekli periyodik heartbeat basması yerine, düğümler varsayılan olarak sessiz (`MUTE`) modda bekleyebilir.

- `DEOS_NODE_MAIN_STM32`, araç hazır duruma geçtiğinde yalnızca izlemekle yükümlü olduğu kritik düğümlere (`Traction`, `BMS`, `Steering`) `HEARTBEAT` servisi için abonelik açar (`Interval = 1000 ms`).
- İzlenmeyen veya yedek bekleyen pasif düğümler hatta paket basmaz.
- Araç acil duruma (`SAFE` / `FAULT`) geçtiğinde abonelikler dinamik olarak kapatılabilir veya frekansları düşürülebilir.

4.7 Değişime Dayalı Telemetri Aboneliği (On-Change / Event-Driven Telemetry)

Düşük bant genişliğine sahip hatlarda (örneğin LoRa telemetrisi), nadir değişen alt sistemlerde (aydınlatma, kontak, konfigürasyon) veya ROS 2 otonom sürüş katmanında, verinin sabit zaman aralıklarıyla (periyodik) basılması yerine yalnızca **durum değiştiğinde** basılması istenir.

Bu amaçla `interval_ms` parametresi için özel bir sabit tanımlanmıştır:

`DEOS_INTERVAL_ON_CHANGE = 0xFFFF (0xFFFF)`

4.7.1 Davranış Kuralları

1. **Zamanlayıcı Baypas Edilir (No Periodic Ticking):** Üretici düğümün 10 ms'lik periyodik tarayıcısı, `interval_ms = 0xFFFF` olan abonelik yuvaları için periyodik tetikleme yapmaz.
2. **Uygulama Tetiklemeli İletim (Event Notification):** Üretici düğümün uygulama katmanı durumunda bir değişiklik tespit ettiğinde (örneğin vites değiştiğinde, farlar açıldığında veya kritik bir eşik aşıldığında) kütüphane API'sini çağırır:

```
deos_subscription_notify_change(DEOS_SERVICE_TRACTION,
    DEOS_CMD_TRACTION_GET_STATUS);
```

   Kütüphane, tablodaki `ON_CHANGE` abonelerini filtreler ve mevcut handler üzerinden güncel durumu anında Unicast olarak ilgili abonelere iletir.
3. **Kira Süresi (Lease TTL) Geçerliliği:** On-Change modunda da `lease_ttl_s` kuralı geçerlidir. Eğer abone düğüm canlı kalmaz ve belirlenen süre içinde aboneliğini yenilemezse, hafızada yetim (orphaned) dinleyici kalmaması için abonelik kaydı otomatik olarak temizlenir.

4.8 C Struct Tanımları

```c
typedef struct
{
    uint8_t  target_service; /* Service ID to subscribe to */
    uint8_t  target_command; /* Status command ID */
    uint16_t interval_ms;    /* Desired interval in ms (0 = unsubscribe) */
    uint16_t lease_ttl_s;    /* Lease duration in seconds (0 = infinite) */
} __attribute__((packed)) deos_subscribe_req_t;

typedef struct
{
    uint8_t  result_code;        /* deos_result_t (0x00 SUCCESS, 0x05 BUSY) */
    uint8_t  target_service;     /* Echo of target service */
    uint8_t  target_command;     /* Echo of target command */
    uint16_t actual_interval_ms; /* Allocated interval in ms */
} __attribute__((packed)) deos_subscribe_resp_t;

typedef struct
{
    uint8_t  target_service; /* Service ID to unsubscribe */
    uint8_t  target_command; /* Command ID (0xFF = all commands) */
} __attribute__((packed)) deos_unsubscribe_req_t;
```